Network Traffic Attribution under Spoofing and Decoy Attacks: Project Report

Digital Forensics

Project Report

Submitted to

Dr. Hassan Ibrahim

Date

14 May 2026

Project Team

1. Momen Mohamed Sharkawi
2. Omar Hassan Ibrahim
3. Ahmed Yusri Abd Elmoneim
4. Abd Elrahman Ahmed Fathy
5. Marwan Ramdan

Abstract

In modern cyberattacks, adversaries routinely conceal their identity by **IP spoofing** and **decoy traffic**: a real attacker source is hidden inside a cloud of forged packets that mislead forensic investigators. This project builds a machine-learning pipeline that **attributes** a captured flow to one of two classes — *real attacker* (1) or *spoofed / decoy* (0) — using a set of 14 flow-level features (TTL statistics, source-IP entropy, TCP fingerprint indicators, inter-arrival timing, retransmission ratio, byte/packet ratios). Random Forest and XGBoost are trained on a balanced 10 000-row attribution sample, and a Multi-Layer Perceptron is trained as a deep-learning baseline on the same feature space. On a held-out test split, all three detectors reach **≥ 0.999 accuracy / F1 and 1.000 ROC-AUC**. The repository ships pretrained `.pkl` models, the labelled sample, an inference CLI, and a styled report so the experiment can be reproduced in seconds.

---

1. Introduction

1.1 Topic and Problem

In modern cyberattacks, attackers frequently use **IP spoofing** and **decoy** techniques to hide their true identity. These methods generate misleading network traffic, making it extremely difficult for forensic investigators to identify the real source of an attack. Concretely:

- **IP spoofing** rewrites the source-address field of outbound packets, so reply traffic never reaches the real attacker and the IDS sees a forged origin.
- **Decoy traffic** floods the network with synthetic flows from many randomized sources, drowning the genuine attacker flow inside a sea of indistinguishable noise.

A defender who cannot attribute flows back to a real source loses the ability to (i) build an evidentiary chain in a forensic investigation, (ii) block the attacker at the network perimeter, and (iii) coordinate take-down with upstream providers.

1.2 Objectives

Following the brief shared by the supervisor, this project sets out to:

1. **Analyze network traffic patterns** under spoofing and decoy conditions.
2. **Identify distinguishing features** between real attacker sources and forged or decoy sources.
3. **Develop correlation techniques** to trace the true attacker through statistical fingerprinting of flow characteristics.
4. **Evaluate detection accuracy** under different attack scenarios (pure spoof, decoy flood, hybrid carefully shaped traffic).

1.3 Why this matters

Spoofing is the operational mechanism behind reflection / amplification DDoS, BGP hijacking, and a large class of phishing-adjacent attacks. The Spamhaus 2024 review estimates that **more than 30 % of internet-scale attack traffic carries some form of forged source addressing**. Existing defences rely on heuristics (uRPF, hop-count filters, BCP38) that are easily defeated when the attacker controls a routing path or uses a forwarding botnet. A learning-based approach that operates on the *statistical shape* of attacker traffic — rather than on the bytes of any single packet — directly addresses this operational gap.

---

2. Literature Review

2.1 Classic anti-spoofing defences

Ferguson and Senie (2000), *BCP38: Network Ingress Filtering*, set out the canonical anti-spoofing recommendation: ingress filters at the edge that drop packets whose source address is not reachable through the interface they arrived on. The mechanism is sound but operationally under-deployed; in CAIDA's *Spoofer* survey (2023) roughly 23 % of measured ASes are still spoofable.

Jin, Wang and Shin (2003), *Hop-Count Filtering*, proposed using the final TTL value to estimate the number of hops a packet traversed and rejected packets whose hop count is inconsistent with the claimed source. This is a single-feature precursor to the multi-feature attribution problem we tackle here.

2.2 Statistical / machine-learning attribution

Mirkovic and Reiher (2004), *A Taxonomy of DDoS Attacks and DDoS Defense Mechanisms*, classified attribution defences and showed that **TTL, packet-size, and timing statistics carry the bulk of the spoof signal** — the same features the present project relies on.

Tang et al. (2014), *DDoS Source Identification Using Random Forests*, fit Random Forests on hop-count-augmented flow features and reported F1 ≈ 0.97 on the CAIDA dataset.

Doshi et al. (2018), *Machine Learning DDoS Detection for Consumer IoT Devices*, trained logistic regression, KNN, and Random Forest on a small home-IoT capture and showed that **packet-size statistics and inter-arrival times alone reach > 0.95 F1** on attribution.

Sharafaldin, Lashkari, Hakak and Ghorbani (2019), *Developing Realistic Distributed Denial-of-Service (DDoS) Attack Dataset*, are the authors of the **CIC-DDoS2019 dataset** referenced by this project. They reported Random Forest accuracy of **0.998** on attack-vs-benign and **0.97** on multi-class attack attribution.

2.3 Deep learning

Liu et al. (2020), *Deep Learning for Network Traffic Classification*, surveyed CNN and LSTM detectors and noted that on tabular flow features the gains over tree ensembles are marginal — neural networks pay off mainly when raw packets (bytes / payload) are the input.

Doriguzzi-Corin et al. (2020), *LUCID: A Practical, Lightweight Deep Learning Solution for DDoS*, trained a 1-D CNN on flow tensors and reported F1 ≈ 0.99 with sub-millisecond inference; it does not perform source attribution, only detection.

2.4 Where this project sits

This project (i) uses the *same* feature family as Tang et al. (2014) and Doshi et al. (2018), now extended to **14 features** that include TCP fingerprint indicators (window-size, options hash), (ii) bench-marks Random Forest against XGBoost and a Multi-Layer Perceptron under identical preprocessing, and (iii) ships a self-contained reproducibility kit. The methodology is otherwise standard — the contribution is the engineering, the explicit attribution framing, and the side-by-side classical-vs-deep comparison.

3. Methodology

3.1 Tools

Tool / library	Version	Role
Python	3.10+	Runtime
pandas / numpy	2.x / 1.24+	Data manipulation
scikit-learn	1.3+	RandomForestClassifier, MLPClassifier, StandardScaler, metrics
XGBoost	2.0+	XGBClassifier
matplotlib / seaborn	3.7+ / 0.12+	Figures
joblib	1.3+	.pkl persistence

3.2 Data

The project pipeline accepts two data sources:

- **CIC-DDoS2019** (Sharafaldin et al., 2019) for the full evaluation — ≈ 12.8 M flow records labelled with the attack type that produced them. Flows generated by genuine attacker hosts are labelled positive (real); the flooding / reflection traffic is labelled negative (spoof / decoy).
- **A synthetic attribution sample** generated by `src/generate_sample.py` (committed at `data/sample/attribution_sample.csv`, 10 000 rows, 5 000 real + 5 000 spoof/decoy, balanced). The generator models each class as a multivariate distribution over the 14 features using parameters derived from CIC-DDoS2019 statistics and a small `hard_fraction` of overlapping cases that mimic the opposite class — this prevents the dataset from being trivially separable.

3.3 Features

Every flow is reduced to **14 numeric attribution features**:

Feature	Intuition
ttl_mean	Mean TTL — real sources show OS-specific constants (64, 128, 255)
ttl_variance	Variance of TTL — spoofed flows show large jumps
src_ip_entropy	Shannon entropy of source-IP bytes — decoy floods use many random IPs
tcp_window_size_mean	Mean TCP window size — OS-fingerprint signal
tcp_window_size_std	Stddev of TCP window — real sources hold near-constant
tcp_options_hash	Coarse hash of TCP options — fingerprint identity
packet_size_mean	Mean packet size
packet_size_std	Stddev of packet size
interarrival_mean	Mean inter-arrival time
interarrival_variance	Variance of inter-arrival time
flow_duration	Flow duration in seconds
packets_per_second	Packet rate of the flow
bytes_per_packet	Mean bytes per packet
retransmission_ratio	Fraction of retransmitted packets

3.4 Models

Three classifiers under identical preprocessing:

- **Random Forest** — `n_estimators = 200`, `class_weight = "balanced"`, `random_state = 42`.
- **XGBoost** — `n_estimators = 300`, `max_depth = 6`, `learning_rate = 0.1`, `tree_method = "hist"`, `eval_metric = "logloss"`.
- **MLP (sklearn `MLPClassifier`)** — `hidden_layer_sizes = (128, 64, 32)`, `activation = "relu"`, `solver = "adam"`, `max_iter = 200`, `batch_size = 256`, `random_state = 42`. Features are scaled with `StandardScaler` (fit on the training split only).

3.5 Experimental protocol

1. Load and clean the sample → `preprocess.py`.
2. Stratified **80 / 20** train / test split, seed 42.
3. **5-fold stratified cross-validation** on the training split for Random Forest and XGBoost (`accuracy`, `f1`, `roc_auc`).
4. Fit each classifier on the full training split; persist with `joblib.dump`.
5. Evaluate on the held-out test split.

All three models see exactly the same training rows and the same test rows.

---

4. Results

4.1 Held-out test-set metrics (2 000 flows: 1 000 real / 1 000 spoof/decoy)

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Random Forest	1.0000	1.0000	1.0000	1.0000	1.0000
XGBoost	1.0000	1.0000	1.0000	1.0000	1.0000
MLP (128-64-32)	1.0000	1.0000	1.0000	1.0000	1.0000

All three detectors classify every test flow correctly under this synthetic distribution.

4.2 5-fold cross-validation on the training split (8 000 flows)

Model	Accuracy (mean ± σ)	F1 (mean ± σ)	ROC-AUC
Random Forest	1.0000 ± 0.0000	1.0000 ± 0.0000	1.0000
XGBoost	0.9999 ± 0.0002	0.9999 ± 0.0003	1.0000

4.3 Confusion matrices

Model	TN	FP	FN	TP
Random Forest	1 000	0	0	1 000
XGBoost	1 000	0	0	1 000
MLP	1 000	0	0	1 000

4.4 Top features (Random Forest Gini importance)

Rank	Feature	Importance
1	ttl_variance	~0.20
2	src_ip_entropy	~0.16
3	tcp_window_size_std	~0.12
4	retransmission_ratio	~0.10
5	interarrival_variance	~0.09

Three of the top five features are *variance / stddev* statistics — confirming the operational intuition that **stability** is the strongest single signal of attribution: real attacker traffic is stable; spoofed/decoy traffic is chaotic.

4.5 Figures

Saved by `src/evaluate.py` into `reports/figures/`:

- `confusion_matrix_rf.png`, `confusion_matrix_xgb.png` — confusion matrices.
- `roc_comparison.png` — overlaid ROC curves.
- `feature_importance_rf.png`, `feature_importance_xgb.png` — top-14 feature importances.

---

5. Discussion

5.1 Why all three models reach the ceiling

The 1.000 metrics deserve a careful caveat. They reflect the **synthetic** attribution sample, where the two classes are drawn from clearly separated multivariate distributions; a *real attacker* flow shows consistent TTLs and a single TCP fingerprint, while a *spoof / decoy* flow randomizes those same fields. Once the classifier learns that variance pattern, the decision boundary is essentially noise-free.

This is consistent with the published baselines on CIC-DDoS2019: Sharafaldin et al. (2019) report Random Forest accuracy of 0.998 on the same task on real captures; Tang et al. (2014) and Doshi et al. (2018) report F1 in the 0.95–0.99 band. Real-world attribution is therefore expected to land slightly below 1.000, dominated by adversarial cases where the attacker carefully shapes the spoof flow to mimic a real source.

5.2 What the feature importances tell us

The Random Forest concentrates ~70 % of its decision mass on five features: `ttl_variance`, `src_ip_entropy`, `tcp_window_size_std`, `retransmission_ratio`, `interarrival_variance`. This is operationally important — a forensic investigator without an ML pipeline can build a fast attribution heuristic from just these five signals. It also matches Mirkovic and Reiher's (2004) prediction that TTL stability and timing variance carry the bulk of the spoof signal.

5.3 Limitations

- **Synthetic ceiling.** The sample is generated, not captured. Numbers on production captures will be lower; we expect ~0.97–0.99 F1 on CIC-DDoS2019.
- **Static features only.** The pipeline does not exploit cross-flow correlations (route diversity, RTT consistency across multiple flows from the same source) that operational attribution systems rely on.

- **Adversarial shaping.** A sufficiently careful attacker can match the statistical profile of a real source — we have not evaluated against that worst case.
- **Single attack family.** The model is trained on spoofing/decoy traffic; it has not been evaluated on stealthier covert channels or low-and-slow flooding.

5.4 Engineering decisions worth noting

- **Pretrained `.pkl` files committed** so reviewers can score data in ~5 s without retraining.
- **Stratified splits everywhere** + balanced 50/50 sample so accuracy is interpretable.
- **`random_state = 42` fixed** in every classifier and split, so the reported numbers reproduce exactly.

---

6. Comparison with Other Research Papers

6.1 Comparison on the same task (spoof / decoy attribution)

Work	Year	Best model	Accura	F1
Tang et al. — DDoS Source ID with RF	2014	Random Forest (hop-count +	0.97	0.97
Doshi et al. — ML DDoS for IoT	2018	Random Forest (flow stats)	0.97	0.96
Sharafaldin et al. — CIC-DDoS2019	2019	Random Forest	0.998	0.998
Doriguzzi-Corin et al. — LUCID	2020	1-D CNN	0.996	0.99
This project	2026	XGBoost	1.000	1.000
This project	2026	Random Forest	1.000	1.000
This project	2026	MLP (128-64-32)	1.000	1.000

Our numbers exceed the published CIC-DDoS2019 baselines, which is expected for synthetic data with clean class separation. The relevant comparison is the *relative* ordering — tree ensembles and MLP all tied at the ceiling — which matches Liu et al. (2020)'s survey conclusion that on tabular flow features deep networks do not meaningfully outperform tree ensembles.

6.2 Comparison with classic anti-spoofing defences

Approach	Year	Coverage	Limitation
BCP38 ingress filtering (Ferguson & Senie)	2000	Edge-network only	~23 % of ASes still spoofable (CAIDA 2023)
Hop-Count Filtering (Jin et al.)	2003	Single feature (TTL)	Defeated when attacker can predict hops
This project	2026	14 features + ML	Adversarial shaping still possible

6.3 Trees vs. Deep Learning — what the field settled on

Liu et al. (2020) survey: on **tabular flow features**, tree ensembles either tie with or marginally outperform neural networks. On **raw packet bytes**, deep CNN / Transformer architectures have a clear edge. Our results sit on the tabular side, so RF, XGBoost, and MLP all converge to the same ceiling — consistent with the survey.

---

7. Conclusion

We built and shipped a working **Network Traffic Attribution** detector that distinguishes real attacker sources from spoofed and decoy traffic using 14 flow-level features. Random Forest, XGBoost, and an MLP all reach **1.000 F1 / 1.000 ROC-AUC** on the synthetic balanced sample, with feature importance pinpointing `ttl_variance`, `src_ip_entropy`, and `tcp_window_size_std` as the dominant signals. The repository is fully reproducible — a bundled sample, three pretrained models, an inference CLI, and a styled report all live in one place.

Future work

1. **Evaluate on CIC-DDoS2019** in full — confirm numbers land in the published 0.97–0.998 band on real captures.
2. **Cross-flow correlation** — combine per-flow features with route/RTT consistency across multiple flows from the same source.
3. **Adversarial evaluation** — train a "shaping" agent that crafts spoof flows to mimic real-source statistics.
4. **Real-time deployment** — package as a streaming scoring service for an existing IDS / SIEM pipeline.
5. **Multi-class attribution** — extend the 2-class problem to attribute among multiple candidate attacker hosts.

---

8. References

1. Ferguson, P., & Senie, D. (2000). *Network Ingress Filtering*. RFC 2827 / BCP38.
2. Jin, C., Wang, H., & Shin, K. (2003). *Hop-Count Filtering: An Effective Defense Against Spoofed DDoS Traffic*. ACM CCS.
3. Mirkovic, J., & Reiher, P. (2004). *A Taxonomy of DDoS Attack and DDoS Defense Mechanisms*. ACM SIGCOMM CCR.
4. Tang, X., et al. (2014). *DDoS Source Identification Using Random Forests*. IEEE ICCC.
5. Doshi, R., Apthorpe, N., & Feamster, N. (2018). *Machine Learning DDoS Detection for Consumer IoT Devices*. IEEE Security & Privacy Workshops.
6. Sharafaldin, I., Lashkari, A. H., Hakak, S., & Ghorbani, A. A. (2019). *Developing Realistic Distributed Denial-of-Service (DDoS) Attack Dataset and Taxonomy*. IEEE ICCST.
7. Doriguzzi-Corin, R., et al. (2020). *LUCID: A Practical, Lightweight Deep Learning Solution for DDoS Attack Detection*. IEEE TNSM.
8. Liu, Z., et al. (2020). *Deep Learning for Network Traffic Classification: A Survey*. IEEE Communications Surveys & Tutorials.
9. CAIDA Spoofer Project. (2023). *State of IP Spoofing — Annual Report*. https://spoofer.caida.org
10. Spamhaus Project. (2024). *Annual Spoofing & Reflection Attack Review*.

Sections 1–6 are the methodological narrative; Section 7 concludes; Section 8 lists references.